Spend Search Where It Pays: Value-Guided Structured Sampling and Optimization for Generative Recommendation

Jie Jiang*
zeus@tencent.com
Tencent Inc.
Beijing, China

Yangru Huang*
yarayrhuang@tencent.com
Tencent Inc.
Beijing, China

Zeyu Wang
peterzywang@tencent.com
Tencent Inc.
Beijing, China

Changping Wang
terracewang@tencent.com
Tencent Inc.
Beijing, China

Yuling Xiong
whitnyxiong@tencent.com
Tencent Inc.
Beijing, China

Jun Zhang†
neoxzhang@tencent.com
Tencent Inc.
Beijing, China

Huan Yu
huanyu@tencent.com
Tencent Inc.
Beijing, China

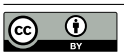
Abstract

Generative recommendation via autoregressive models has unified retrieval and ranking into a single conditional generation framework. However, fine-tuning these models with Reinforcement Learning (RL) often suffers from a fundamental **probability-reward mismatch**. Conventional likelihood-dominated decoding (e.g., beam search) exhibits a myopic bias toward locally probable prefixes, which causes two critical failures: (1) *insufficient exploration*, where high-reward items in low-probability branches are prematurely pruned and rarely sampled, and (2) *advantage compression*, where trajectories sharing high-probability prefixes receive highly correlated rewards with low within-group variance, yielding a weak comparative signal for RL. To address these challenges, we propose **V-STAR**, a **V**alue-guided **S**ampling and **T**ree-structured **A**dvantage **R**einforcement framework. V-STAR forms a self-evolving loop via two synergistic components. First, a **Value-Guided Efficient Decoding (VED)** is developed to identify decisive nodes and selectively deepen high-potential prefixes. This improves exploration efficiency without exhaustive tree search. Second, we propose **Sibling-GRPO**, which exploits the induced tree topology to compute sibling-relative advantages and concentrates learning signals on decisive branching decisions. Extensive experiments on both offline and online datasets demonstrate that V-STAR outperforms state-of-the-art baselines, delivering superior accuracy and candidate-set diversity under strict latency constraints.

CCS Concepts

• **Information systems** → **Recommender systems**.

*Equal contribution.
†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD 2026, Jeju Island, Republic of Korea.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817873

Keywords

Generative Recommendation; Reinforcement Learning; Value-Guided Efficient Decoding; Sibling-GRPO

ACM Reference Format:
Jie Jiang, Yangru Huang, Zeyu Wang, Changping Wang, Yuling Xiong, Jun Zhang, and Huan Yu. 2026. Spend Search Where It Pays: Value-Guided Structured Sampling and Optimization for Generative Recommendation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817873

1 Introduction

Recommender systems are shifting from retrieve-and-rank to end-to-end generative recommendation (GR) [21, 26, 49], which has been deployed in e-commerce and short-video platforms [26, 49]. By reformulating item identifiers into a hierarchical search space of Semantic IDs (SIDs), large-scale candidate generation has become computationally practical with autoregressive language models [10, 37, 38, 45]. To further bridge the gap between generation likelihood and real-world utility, these GR models are usually fine-tuned with reinforcement learning (RL) [5, 7, 30] techniques, e.g., on-policy GRPO-style groupwise updates [10, 15, 20, 42, 51]. However, such a training mechanism is bottlenecked by a fundamental probability-reward misalignment. During training, candidate items are typically sampled by probability-driven decoding strategies (e.g., beam search). In contrast, the recommendation objective aims to maximize the expected reward of the full generated SIDs.

This misalignment creates a tension between the greedy nature of generation and the long-term goal of exploration, manifesting in two critical structural failures: (1) **insufficient exploration of high-reward items:** Many high-reward items begin with lower-probability tokens because they have limited historical interactions. Standard beam search prunes branches based on likelihood and can irreversibly discard such paths early in generation. Consequently, the model becomes blind to high-reward candidates simply because their prefixes are not immediately likely. (2) **advantage**

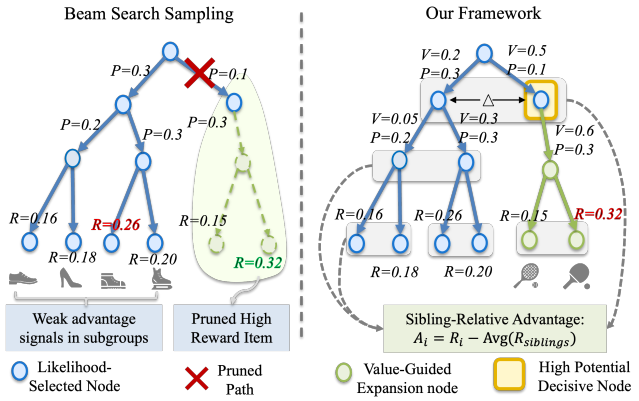


Figure 1: Comparison of Beams Search and V-STAR. Left: probability-based pruning removes high-reward items and produces homogeneous candidates. Right: V-STAR expands high-value prefixes under a fixed budget and strengthens within-group learning with Sibling-GRPO.

compression within the generated group: Guided by likelihood, decoding tends to over-expand multiple branches that share high-probability prefixes, producing a candidate set dominated by siblings, i.e., near-duplicate items with similar rewards. However, effective RL relies on reward contrast: policy gradients are driven by differences between good and bad actions. In such sibling-heavy groups, rewards become highly correlated and concentrate in a narrow band, while a few outliers can set the overall reward scale. As a result, GRPO's group-wise normalization is dominated by the global reward range and shrinks the relative advantage differences among siblings, weakening the learning signal at the decisions that actually matter.

Previous methods attempt to mitigate these issues, but each incurs a hard trade-off. One line of work increases the sampling budget [12, 41, 44], yet indiscriminate expansion largely spends compute on redundant, high-probability candidates. Another line relies on heuristic sampling (e.g., temperature scaling or nucleus sampling) to increase sample diversity [14, 24, 36], but the added randomness is difficult to control, often hurting relevance and making high-reward outcomes less reliable. More structured approaches, such as Monte Carlo Tree Search or tree-of-thought style search [23, 32, 46], can improve exploration in LLM inference, but their computational cost is prohibitive for broad use in recommendation. A critical gap therefore remains: we lack a mechanism that reliably determines *when* exploration is worth paying for and *where* limited compute should be allocated to maximize reward.

Motivated by this gap, we make a simple observation: under a fixed decoding budget, extra exploration does *not* help equally at every generation step. Many prefixes are clearly low-potential, or their next token is already obvious, so additional compute there is largely wasted. In contrast, exploration matters most at a small number of *decisive* prefixes where several plausible branches compete and which branch to choose strongly affects the final reward (e.g., whether long-tail but high-reward items can be reached). We detect these decisive points using two signals: (1) **high value**, meaning the prefix is promising in expected reward, and (2) **high ambiguity given high value**, meaning the next-step choice is uncertain enough that extra search can change the outcome. This perspective turns exploration into a targeted budgeting problem: decide *where* to spend compute so that it most improves reward, which directly motivates a value-guided decoding strategy and a tree-structured credit assignment objective.

Building on these insights, we propose **V-STAR** (Value-guided Sampling and Tree-structured Advantage Reinforcement), a self-evolving sampling-and-learning framework where better sampling provides clearer training signals, and the resulting updates further improve sampling. V-STAR comprises two key components: **Value-Guided Efficient Decoding (VED)** for candidate sampling and **Sibling-GRPO** for tree-structured policy optimization. **VED** tackles *insufficient exploration* by performing budgeted value-guided expansion on the SID prefix tree. It initializes a shallow prefix tree with likelihood-guided beam search to cheaply reveal where hypotheses concentrate and where promising branches are pruned early. It then combines a lightweight value estimator with an uncertainty signal (e.g., policy entropy) to select a small set of high-potential prefixes and spends additional compute only at these divergence points, enabling local expansion and backup under strict depth and branching constraints. This targeted allocation corrects premature pruning and improves the reachability of long-tail, high-reward items without incurring exhaustive tree search. **Sibling-GRPO** addresses *advantage compression* in prefix-coupled generation by introducing a structured group-relative objective over genealogically related candidates. Rather than applying a single global normalization over the entire candidate set, it forms sibling groups under each parent prefix and performs within-group relative advantage learning. This concentrates gradients on the decisive branching actions where candidates diverge yet remain highly correlated, mitigating advantage compression and recovering informative learning signals inside homogenized candidate clusters. Together, VED improves candidate quality and diversity, while Sibling-GRPO turns these gains into more stable and informative updates.

In summary, our contributions are as follows:

(1) We formalize probability–reward misalignment in generative recommendation and analyze its impact on sample decoding and RL fine-tuning. Building on this analysis, we propose V-STAR to address the issue via value-guided decoding and tree-structured credit assignment.
(2) We develop Value-Guided Efficient Decoding (VED) for budgeted candidate construction by allocating decoding compute to high-value and high-ambiguity nodes.
(3) We introduce Sibling-GRPO, a tree-structured objective that uses sibling groups to mitigate advantage compression under correlated candidates.
(4) Experiments on both offline datasets and online settings show consistent gains over strong generative baselines.

2 Related Work

2.1 Generative Recommender

The paradigm of recommender systems is shifting from discriminative matching to generative modeling. Early works linearized user

histories and predicted the next item ID directly, but faced challenges due to the extremely large item-ID vocabulary [9, 33]. To mitigate this issue, recent approaches such as P5 [11] and TIGER [26] introduce SID, which represent each item as a short sequence of hierarchical discrete tokens constructed via residual quantization (RQ-VAE) [47] or collaborative indexing. This formulation bridges recommendation and language modeling, enabling Transformer architectures to generate items token by token. Most existing SID-based recommenders still rely on likelihood-driven decoding (e.g., beam search) at training time. Recent analyses [22] show that beam search exhibits a rich-get-richer bias and can prune low-probability branches early in generation. In recommendation, high-reward items (e.g., niche discoveries) may begin with low-probability prefix tokens, so early prefix pruning can make a substantial portion of the item space effectively unreachable. This is a structural limitation that is not resolved by model scaling alone.

2.2 Decoding Strategies in Generation

Decoding (sampling) strategies control the trade-off between exploration and exploitation. Heuristic sampling methods (e.g., Top-K and nucleus sampling [14]) introduce stochasticity to improve diversity, but they provide no explicit mechanism to guarantee reward and can lead to irrelevant recommendations. In contrast, tree-search algorithms such as Monte Carlo Tree Search (MCTS) and Tree of Thoughts (ToT) [46] enable lookahead and backtracking and have shown strong performance on multi-step reasoning tasks. However, these methods are often impractical for industrial recommendation because strict latency budgets make per-request rollouts prohibitively expensive. Some works distill search behavior into a policy [1], but they typically treat search as a black-box optimizer. Our work instead proposes a budget-aware sparse search that dynamically allocates a finite compute budget to a small set of pivotal nodes, capturing the benefits of lookahead without the overhead of full tree expansion.

2.3 RL for Generative Alignment

RL is widely used to align generative models with non-differentiable objectives (e.g., CTR and diversity). Common approaches include REINFORCE [40], PPO [28], GRPO [29], and recent direct preference-optimization methods such as DPO [25]. In recommendation, RL has been applied to optimize slate generation (e.g., SlateQ [17]) and to correct exposure bias [6]. However, existing generative RL methods often implicitly treat candidate samples as independent and identically distributed (i.i.d.) [10, 38]. In autoregressive generation, candidates frequently share long prefixes and therefore form sibling groups with highly correlated features and rewards. Standard policy-gradient updates can be ineffective in this setting: subtracting a baseline from a set of nearly identical rewards yields a weak and noisy advantage signal, which can lead to advantage collapse [27]. Our proposed Sibling-GRPO addresses this issue by explicitly modeling relative advantages within sibling groups, which is related in spirit to listwise ranking objectives [4] but tailored to the hierarchical structure of the generation tree.

3 Preliminaries

3.1 Problem Formulation

Generative Recommendation with Semantic IDs. We formulate recommendation as a conditional sequence generation task [11, 34]. Under the SID paradigm [26], each item is represented by a unique, fixed-length sequence of discrete tokens $y = (y_1, \ldots, y_L) \in \mathcal{V}^L$, where $\mathcal{V}$ is a finite token vocabulary and L is the (fixed) SID length. Given a user context x (e.g., interaction history and profile features), an autoregressive policy π_θ models the conditional probability of an item sequence via the probability chain rule: $\pi_\theta(y \mid x) = \prod_{l=1}^{L} \pi_\theta(y_l \mid x, y_{\le l-1})$, where $y_{\le l-1} = (y_1, \ldots, y_{l-1})$ denotes the already generated prefix before step l, with $y_{\le 0} = \varnothing$ (empty prefix, i.e., generation starts from the user context x alone).
Decoding Operator. We abstract decoding process as an operator $\mathcal{D}$ that maps policy π_θ and context x to a candidate set $C(x)$:

$$C(x) = \mathcal{D}(\pi_\theta, x), \qquad C(x) \subset \mathcal{V}^L. \tag{1}$$

Ideally, $\mathcal{D}$ should return candidates that maximize the ground-truth reward $R(x, y)$. In practice, standard decoding strategies (e.g., beam search [44]) approximate this objective by maximizing model probability $\pi_\theta(y \mid x)$ through heuristic prefix pruning.
Policy Refinement via GRPO. To align the pre-trained policy with non-differentiable ranking objectives, we apply GRPO [29] to the decoded set $C(x)$ as the empirical reference set for reward comparison. The standard group-normalized advantage is used:

$$A(x, y) = \frac{R(x, y) - \mu_R(x)}{\sigma_R(x) + \epsilon}, \tag{2}$$

where $\mu_R(x)$ and $\sigma_R(x)$ are the mean and standard deviation of rewards over $C(x)$ and $\epsilon > 0$ ensures numerical stability. In addition to standard GRPO, our method also applies sibling GRPO to further enhance learning signals, which will be introduced in Sec. 4.3.

3.2 Probability-Driven Decoding Bias

Insufficient exploration on high-reward items. Autoregressive decoding process can be viewed as traversing a prefix tree [39]: SIDs sharing a common prefix follow the same branch until they diverge. Under a fixed budget, pruning-based methods [8, 24, 36] (e.g., beam search or top-K sampling) keep only a small set of active prefixes and discard the rest by local likelihood. Pruning is irreversible in this case: once a prefix is removed, all items under that branch will never be evaluated. This is problematic because likelihood and reward can be misaligned: there may exist y_a, y_b with $\pi_\theta(y_a \mid x) > \pi_\theta(y_b \mid x)$ but $R(x, y_a) < R(x, y_b)$. As a result, likelihood-only pruning may eliminate low-probability prefixes that lead to high-reward items, and those branches are rarely revisited.
Advantage compression within the generated group. Given a candidate set $C(x)$, we define the within-group reward range as:

$$\Delta_R(x) \triangleq \max_{y \in C(x)} R(x, y) - \min_{y \in C(x)} R(x, y). \tag{3}$$

Since $|R(x, y) - \mu_R(x)| \le \Delta_R(x)$ for all $y \in C(x)$, the within-group advantage magnitude is bounded by:

$$|A(x, y)| = \left| \frac{R(x, y) - \mu_R(x)}{\sigma_R(x) + \epsilon} \right| \le \frac{\Delta_R(x)}{\sigma_R(x) + \epsilon} \le \frac{\Delta_R(x)}{\epsilon}. \tag{4}$$

Eq. 4 implies that $A(x,y) \in [-\frac{\Delta_R(x)}{\epsilon}, \frac{\Delta_R(x)}{\epsilon}]$. By Popoviciu's inequality on variances, $\mathrm{Var}_{y\in C(x)}[A(x,y)] \le (\frac{\Delta_R(x)}{\epsilon})^2$, so the standard deviation $\sigma_A(x) \triangleq \sqrt{\mathrm{Var}_{y\in C(x)}[A(x,y)]} \le \frac{\Delta_R(x)}{\epsilon}$. Therefore, when candidate collapse in $C(x)$ yields small $\Delta_R(x)$, it forces $\sigma_A(x)$ to be small, making advantages nearly indistinguishable. We refer to this degeneration of learning signal as *advantage compression*.

Implication. Together, insufficient exploration and advantage compression show that decoding is not a mere inference-time heuristic [31]: it controls both *reachability* (which high-reward items are ever surfaced) and *learnability* (whether candidates provide enough reward contrast for RL). We therefore need a decoder that avoids irreversible pruning of low-probability yet high-reward branches and yields candidate groups with sufficient reward dispersion, motivating our dynamic decoding framework.

4 Methodology

Our V-STAR (Fig. 2) is a unified decoding-and-learning framework consisting *Value-Guided Efficient Decoding* (VED) for candidate construction and *Sibling-GRPO* for policy optimization. VED addresses *reachability* by allocating compute to decisive prefixes under a fixed budget, while Sibling-GRPO addresses *learnability* by restoring within-group learning signal under prefix coupling.

4.1 Semantic-aware Value Model Learning

VED relies on lookahead value estimation to mitigate probability-driven bias on the SID prefix tree. To this end, we first learn a prefix value function V_ϕ that maps each prefix to its expected downstream return [28]. At generation step $\ell \in \{1,\dots,L\}$, we define the decoding state s_ℓ as $s_\ell \triangleq (x, y_{\le\ell})$, where x is the user context and $y_{\le\ell} = (y_1,\dots,y_\ell)$ is the current SID prefix. The value function estimates the discounted return of continuing generation from s_ℓ:

$$V_\phi(s_\ell) \triangleq \mathbb{E}\left[\sum_{t=\ell}^{L} \gamma^{t-\ell} r_t \,\middle|\, s_\ell\right], \tag{5}$$

where $\gamma \in (0,1]$ is the discount factor. We parameterize V_ϕ as a lightweight value head on top of the policy backbone: a single shallow Transformer block (operating on the prefix hidden states) followed by an MLP regressor, enabling frequent value queries with negligible overhead.

Semantic-aware dense supervision. Learning V_ϕ from sparse terminal returns (non-zero only when the generated item exactly matches the ground truth) provides weak, delayed signals for intermediate prefixes. We therefore build dense semantic returns from pre-computed item embeddings, obtained by encoding each item's textual description with a frozen text encoder. Given context x, let $C(x) = \mathcal{D}(\pi_\theta, x)$ be the sampled candidate set. For prefix $y_{\le\ell}$, we define the *sampled prefix bucket* to be the subset of items in $C(x)$ that shares this prefix:

$$C(x; y_{\le\ell}) \triangleq \left\{ y' \in C(x) \mid y'_{\le\ell} = y_{\le\ell} \right\}, \tag{6}$$

where $y'_{\le\ell}$ denotes the SID prefix of the candidate item y'. We then define a context-conditional *prefix embedding* by averaging embeddings over this bucket:

$$\bar{\mathbf{e}}(x, y_{\le\ell}) \triangleq \frac{1}{|C(x; y_{\le\ell})|} \sum_{i \in C(x; y_{\le\ell})} \mathbf{e}(i), \tag{7}$$

where $\mathbf{e}(i)$ is the embedding of candidate item i. The step-wise dense return r_ℓ for prefix $y_{\le\ell}$ is then defined as:

$$r_\ell = \begin{cases} w_\ell, & \text{if } y_{\le\ell} = y^\star_{\le\ell}, \\ -w_\ell\big(1 - \cos\big(\bar{\mathbf{e}}(x, y_{\le\ell}), \mathbf{e}(y^\star)\big)\big), & \text{otherwise,} \end{cases} \tag{8}$$

where $w_\ell > 0$ is a monotonically increasing weight with respect to ℓ, cos denotes the cosine similarity between the embeddings, and $\mathbf{e}(y^\star)$ represents the embedding of the ground-truth item $y^\star$ for context x. Compared to sparse exact-match supervision, the proposed semantic-aware dense supervision provide informative feedback on mismatched prefixes by leveraging their semantic proximity to the ground-truth item, thereby improving value estimation for intermediate decoding states.

Temporal-difference learning. Given the stepwise signal in (8), we train the value function by one-step temporal-difference learning, enforcing Bellman-style consistency across successive prefixes. Concretely, the TD target and regression loss are:

$$\tilde{y}_\ell = \begin{cases} r_\ell + \gamma V_\phi(s_{\ell+1}), & \ell < L, \\ r_L, & \ell = L, \end{cases} \qquad \mathcal{L}_V(\phi) = \mathbb{E}\Big[\big(V_\phi(s_\ell) - \tilde{y}_\ell\big)^2\Big], \tag{9}$$

where $s_{\ell+1} = (x, y_{\le\ell+1})$. The resulting V_ϕ provides an efficient, reward-aligned lookahead over prefixes, which is later used to guide budgeted expansion toward decisive prefix nodes in Sec. 4.2.

4.2 Value-Guided Efficient Decoding

With the value model as guidance, we cast semantic-ID decoding as constructing a candidate set under a strict compute budget on the SID prefix tree. Given context x, the decoder returns $C(x) \subset \mathcal{V}^L$ that (i) contains high-reward items and (ii) enables informative within-group comparisons for groupwise RL.

A budgeted objective for candidate sets. Let $\mathrm{Cost}(C(x))$ denote the compute required to construct $C(x)$, measured by the number of backbone forward tokens incurred during decoding. We formulate candidate construction as a constrained set optimization problem:

$$\max_{C(x)\subseteq\mathcal{V}^L} \underbrace{\mathbb{E}_{y\in C(x)}[R(x,y)]}_{\text{reward}} + \lambda \underbrace{\mathrm{Contrast}(C(x))}_{\text{dispersion}} \quad \text{s.t.} \quad \mathrm{Cost}(C(x)) \le B. \tag{10}$$

where $\mathrm{Contrast}(\cdot)$ is a set-level regularizer that encourages within-group discriminability, e.g., by increasing reward dispersion and mitigating prefix coupling (Sec. 3.2). The budget B captures the computational constraint in recommendation.

Value-based scoring with entropy regularization. Directly optimizing Eq. (10) is intractable due to the exponential SID space. VED therefore allocates the decoding budget to a small set of *decisive prefixes*. The key question is how to rank prefix states $s = (x, y_{\le\ell})$ for further expansion. To maximize the expected improvement per unit cost, prefix expansion should balance *exploitation* and *exploration*. We therefore define a prefix priority score:

$$G(s) \triangleq \begin{cases} V_\phi(s) + \lambda \mathcal{H}_\theta(s), & \ell < L, \\ V_\phi(s), & \ell = L, \end{cases} \tag{11}$$

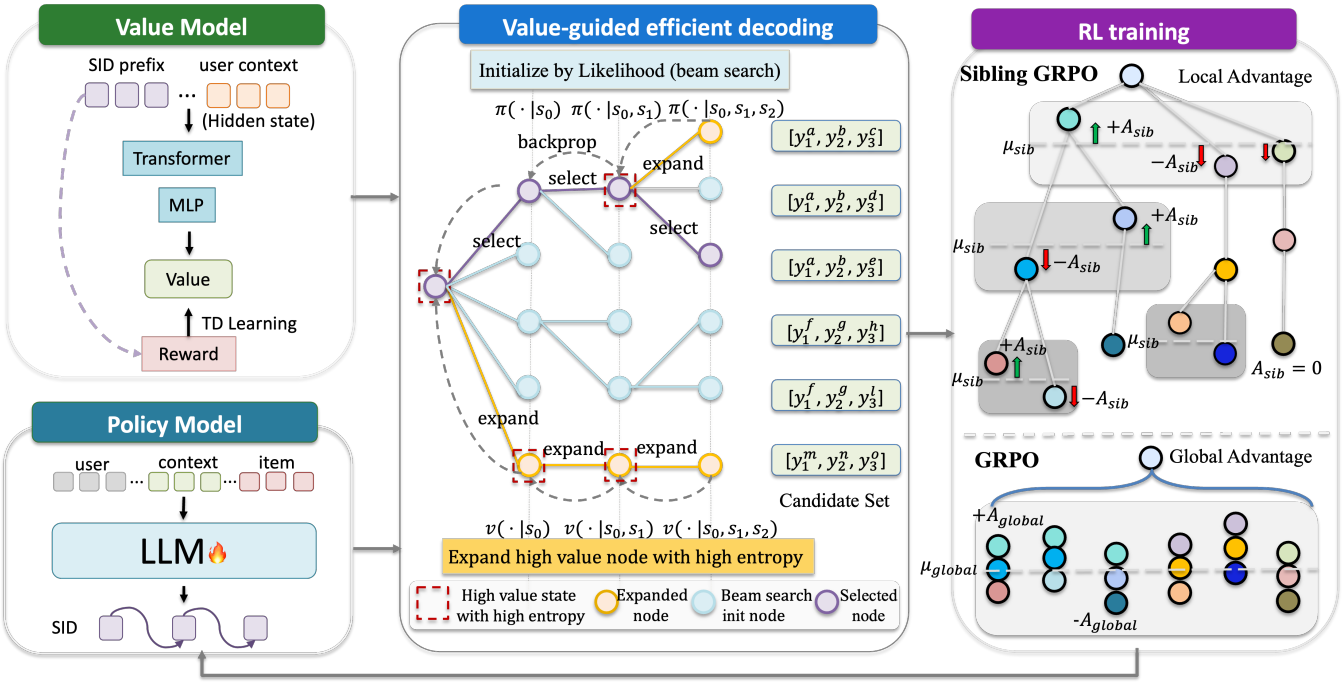


Figure 2: Overview of the proposed self-evolving decoding-and-learning framework V-STAR.

where $V_\phi(s)$ is the predicted downstream return from prefix s, λ is the regularization weight, and $\mathcal{H}_\theta(s)$ is the entropy term that measures how uncertain the policy is about the next token:

$$\mathcal{H}_\theta(s) \triangleq -\sum_{y_{\ell+1}\in\mathcal{V}} \pi_\theta(y_{\ell+1} \mid x, y_{\le\ell}) \log \pi_\theta(y_{\ell+1} \mid x, y_{\le\ell}). \quad (12)$$

For terminal prefixes ($\ell = L$), the priority reduces to $V\phi(s)$.

Score-based efficient decoding. With the above value-based prefix score, the decoding process performs budgeted search and expansion on a search tree $\mathcal{T}$ over prefix states $s = (x, y_{\le\ell})$. The procedure consists of four stages: ① initialization, ② selection and traversal, ③ gated expansion, and ④ backpropagation.

① *Initialization.* We initialize $\mathcal{T}$ using prefixes obtained by a low-cost probability-guided beam search under π_θ. The beam width in this warm start is set smaller than the final candidate set size to limit overhead. The resulting prefix tree reveals which prefixes the policy prefers and which branches are pruned early. We evaluate each node in the initial tree once to populate $\{V_\phi(s), \mathcal{H}_\theta(s), G(s)\}$ and record whether the node admits unexpanded valid children.

② *Selection and traversal.* After initialization, we repeatedly select nodes and traverse a root-to-leaf path. Let $N(s)$ be the visit count of node s and N_{root} the number of traversals so far, we select nodes with a UCB-style score [19]:

$$U(s) \triangleq G(s) + \beta \cdot \sqrt{\frac{\ln(N_{\text{root}}+1)}{N(s)+1}}, \quad (13)$$

where β controls exploration. Starting from the root, we recursively choose the child with the largest $U(s)$ until reaching a leaf or terminal node.

③ *Gated expansion.* Along the selected path, we expand only prefixes that are decisive relative to other prefixes at the same depth. Let $\mathcal{T}_\ell$ denote the set of nodes in the current tree at depth ℓ, and define the depth-wise average priority

$$\bar{G}_\ell \triangleq \frac{1}{|\mathcal{T}_\ell|} \sum_{u\in\mathcal{T}_\ell} G(u). \quad (14)$$

For any visited node $s = (x, y_{\le\ell})$ on the path that is not fully expanded, we trigger one-step expansion if and only if $G(s) \ge \bar{G}_\ell$. When triggered, we add one new child to s by sampling a token from its yet-unexpanded valid children according to their normalized policy probabilities.

④ *Backpropagation.* For each newly added child s_{new}, we evaluate $(V_\phi, \mathcal{H}_\theta, G)$ and update statistics along its ancestor chain, including visit count $N(\cdot)$ and depth-wise average priority scores.

The above stages ②–④ are repeated until $\text{Cost}(\mathcal{T}) > B$, where $\text{Cost}(\cdot)$ counts the number of backbone forward tokens. After termination, we extract a candidate set $\mathcal{C}(x) \subset \mathcal{V}^L$ from the search tree $\mathcal{T}$ in a value-aware manner. In practice, we sample top-valued SIDs from depth-L leaf nodes. In rare cases, if insufficient leaf nodes exist, we fill the remainder by completing high-value prefixes in the tree using cached π_θ, without additional backbone forwards. The resultant candidate set $\mathcal{C}$ improves reachability by exploring low-probability but high-value prefixes.

4.3 Sibling-GRPO

While VED improves reachability and increases candidate reward dispersion, hierarchical SIDs still induce common-prefix sibling node groups in $\mathcal{C}(x)$. Sibling-GRPO exploits these groups to recover strong learning signals exactly at the decisive branching actions.

Table 1: Performance comparison on Industrial and Office. Gen. means Generative model. Best results are bolded.

Model	Gen.	SID	RL	Industrial						Office					
				HR@3	HR@5	HR@10	NDCG@3	NDCG@5	NDCG@10	HR@3	HR@5	HR@10	NDCG@3	NDCG@5	NDCG@10
GRU4Rec	–	–	–	0.0638	0.0774	0.0999	0.0542	0.0598	0.0669	0.0629	0.0789	0.1019	0.0528	0.0595	0.0669
Caser	–	–	–	0.0618	0.0717	0.0942	0.0514	0.0555	0.0628	0.0748	0.0865	0.1093	0.0615	0.0664	0.0737
SASRec	–	–	–	0.0790	0.0909	0.1088	0.0700	0.0748	0.0806	0.0861	0.0949	0.1120	0.0769	0.0805	0.0858
HSTU	✓	–	–	0.0927	0.1037	0.1163	0.0885	0.0918	0.0958	0.1134	0.1252	0.1400	0.1031	0.1079	0.1126
BIGRec	✓	–	–	0.0931	0.1092	0.1370	0.0841	0.0907	0.0997	0.1069	0.1204	0.1434	0.0961	0.1017	0.1091
TIGER	✓	✓	–	0.0852	0.1010	0.1321	0.0742	0.0807	0.0908	0.0986	0.1163	0.1408	0.0852	0.0960	0.1002
LCRec	✓	✓	–	0.0915	0.1057	0.1332	0.0805	0.0862	0.0952	0.0921	0.1048	0.1237	0.0807	0.0859	0.0920
D3	✓	✓	–	0.1024	0.1213	0.1500	0.0991	0.0989	0.1082	0.1204	0.1406	0.1634	0.1055	0.1139	0.1213
S-DPO	✓	✓	✓	0.1032	0.1238	0.1524	0.0906	0.0991	0.1082	0.1169	0.1356	0.1587	0.1033	0.1110	0.1255
MiniOneRec	✓	✓	✓	0.1143	0.1321	0.1586	0.1011	0.1084	0.1167	0.1217	0.1420	0.1634	0.1088	0.1172	0.1242
Ours-Train	✓	✓	✓	**0.1189**	**0.1361**	**0.1641**	**0.1057**	**0.1128**	**0.1217**	**0.1344**	**0.1500**	**0.1746**	**0.1196**	**0.1260**	**0.1340**

Sibling groups and sibling nodes. An SID $y = (y_1, \ldots, y_L)$ corresponds to a root-to-leaf path on a depth-L prefix tree. Fix a depth $\ell \in \{1, \ldots, L\}$ and let $h \in \mathcal{V}^{\ell-1}$ denote a parent prefix at depth $\ell - 1$. We first define the *sibling group* under h as the sampled items that share this parent prefix:

$$\mathcal{G}(h) \triangleq \{ y \in C(x) \mid y_{\leq \ell-1} = h \}. \tag{15}$$

Next, we define the *sibling node set* under h as the set of child nodes at depth ℓ that extend h by one token:

$$\mathcal{S}(h) \triangleq \{ v \mid v \in \mathcal{V},\ \exists y \in \mathcal{G}(h) \text{ s.t. } y_\ell = v \}. \tag{16}$$

Relative advantages over sibling nodes. Sibling-GRPO is computed within $\mathcal{S}(h)$ by assigning each child node $v \in \mathcal{S}(h)$ a node-level score defined as the average reward of candidates routed through that child:

$$\begin{aligned} \bar{R}(x; h, v) &\triangleq \frac{1}{|\mathcal{G}(h, v)|} \sum_{y \in \mathcal{G}(h,v)} R(x, y), \\ \mathcal{G}(h, v) &\triangleq \{ y \in \mathcal{G}(h) \mid y_\ell = v \}. \end{aligned} \tag{17}$$

This construction yields localized, sibling-level comparisons that focus learning on the branching decision at depth ℓ. Let $\mu_h(x)$ and $\sigma_h(x)$ denote the mean and standard deviation of $\{\bar{R}(x; h, v)\}_{v \in \mathcal{S}(h)}$, the sibling-relative node advantage for a child $v \in \mathcal{S}(h)$ is

$$A_{\text{node}}(x; h, v) = \frac{\bar{R}(x; h, v) - \mu_h(x)}{\sigma_h(x) + \epsilon}. \tag{18}$$

Compared with global normalization over $C(x)$, this sibling-node advantage prevents within-parent advantages from being washed out by coarse variations across unrelated prefixes, and it concentrates learning signals on the branching action y_ℓ that differentiates siblings under the same parent prefix.

Objective. Sibling-GRPO retains the GRPO update form but applies it to the sibling nodes within each depth. For a parent prefix $h \in \mathcal{V}^{\ell-1}$ at depth $\ell - 1$ and child node $v \in \mathcal{S}(h)$. We define the token-level importance ratio as $\rho_\theta(v \mid x, h) \triangleq \frac{\pi_\theta(v|x,h)}{\pi_{\theta_{\text{old}}}(v|x,h)}$, where $\pi_{\theta_{\text{old}}}$ denotes the frozen behavior policy used to sample $C(x)$. We then maximize the following sibling-node GRPO objective aggregated over depths:

$$\mathcal{J}_{\text{sib}}(\theta) = \mathbb{E}_x \left[\frac{1}{|C(x)|} \sum_{\ell=1}^{L} \sum_{h} \sum_{v \in \mathcal{S}(h)} A_{\text{node}}(x; h, v)\, \rho_\theta(v \mid x, h) \right]. \tag{19}$$

This formulation performs GRPO-style [29] updates on branching tokens at each depth, with sibling-normalized advantages computed across competing child nodes under the same parent prefix.

4.4 Training and Testing

Training-Time Self-Evolution. Our V-STAR primarily focuses on *training-time* improvement. During training, VED and Sibling-GRPO form a closed loop: VED uses the value V_ϕ together with the entropy signal to selectively refine search and generate structured candidate groups with better reachability and clearer reward contrast. Sibling-GRPO then converts these structured groups into stable, high-quality policy updates by comparing sibling alternatives at each branching level. As training progresses, both π_θ and V_ϕ improve, which sharpens the acquisition score in Eq. (11) and further improves subsequent candidate construction—yielding a self-evolving decoding-and-learning loop.

Testing-Time Decoding. At test time, we use standard beam search [34] decoding for efficiency and compatibility with existing serving pipelines [10]. Optionally, decoding with VED can also be applied at inference: the learned V_ϕ and the same acquisition rule Eq. (11) guide the decoder to select a small number of high-potential prefixes under a fixed compute budget, improving long-tail coverage and candidate diversity when additional search is allowed.

5 Experiments

5.1 Experimental Setup

Datasets and Evaluation. Experiments are conducted on both offline public dataset and online data. For offline evaluation, we use Amazon Review dataset [16] with two subsets *Industrial* and *Office Products*. Following the standard next-item prediction protocol, interactions are ordered chronologically per user. The model

Table 2: Ablation on decoding strategies.

Decoding	Industrial		Office	
Strategy	NDCG@10 ↑	HR@10 ↑	NDCG@10 ↑	HR@10 ↑
Beam Search	0.1194	0.1606	0.1299	0.1684
Top-K	0.1090	0.1538	0.1226	0.1644
Ours	**0.1217**	**0.1641**	**0.1340**	**0.1746**

Table 3: Ablation of expand rules in VED.

Expansion	Industrial		Office	
Rule	NDCG@10 ↑	HR@10 ↑	NDCG@10 ↑	HR@10 ↑
$V_\phi(s)$	0.1202	0.1627	0.1333	0.1742
$\mathcal{H}_\theta(s)$	0.1198	0.1604	0.1259	0.1688
$G(s)$	**0.1217**	**0.1641**	**0.1340**	**0.1746**

takes the entire historical sequence as input and predicts the next interacted item. For data preprocessing, we follow the procedure described in [20]. Each item is represented by a 3-level SID derived from the RQ-VAE codebook [26], and all generative decoders operate under a hard constraint that restricts outputs to valid SIDs, ensuring that each generated sequence maps to a unique catalog item. We report standard metrics Hit Rate (HR) and Normalized Discounted Cumulative Gain (NDCG). HR@K measures whether the ground-truth next item appears in the Top-K list, while NDCG@K further accounts for its ranked position. Unless otherwise specified, we set $K = 3, 5, 10$ in the main tables. For online evaluation, we use real traffic data, which will be detailed in Sec. 5.3.

Implementation Details. We use Qwen2.5-1.5B [43] as the model backbone. All models are trained in two stages: supervised fine-tuning (SFT) followed by RL with GRPO. SFT is run with data parallelism on 8 GPUs with a global batch size of 1024 and a fixed random seed. RL is trained on 8 GPUs with a per-GPU batch size 64 and gradient accumulation step 2. The learning rate is set to 1×10^{-5}, and KL regularization coefficient is 10^{-3}. During training, VED is utilized to construct 16 candidates per query, with a prefix length $L = 3$ and hierarchical weights $w_l = [0.3, 0.5, 1.0]$. The discount factor γ in Eq. 9 is set to 0.99 and the exploration coefficient λ in Eq. 11 is set to 0.1. The beam width for VED prefix-tree initialization is set to 8. To ensure serving efficiency, we adopt standard beam search as the default decoder during inference, effectively decoupling the exploration-intensive training from latency-critical deployment.

5.2 Performance on Offline Dataset

The proposed method is compared with recent state-of-the-art baselines under the same setting, including traditional embedding-based sequential recommenders (GRU4Rec [13], Caser [35], and SASRec [18]), generative non-SID baselines (HSTU [48], BIGRec [2]), SID-based generative recommenders without RL-style optimization (TIGER [26], LCRec [50], D3 [3]), and SID-based methods with RL or preference optimization (S-DPO [7], MiniOneRec [20]). Table 1 summarizes results on *Industrial* and *Office*. Our method consistently achieve the highest performance among all methods. Compared with the strongest RL baseline MiniOneRec, our method

Table 4: Ablation on training objective.

Training	Industrial		Office	
Objective	NDCG@10 ↑	HR@10 ↑	NDCG@10 ↑	HR@10 ↑
GRPO	0.1189	0.1598	0.1302	0.1712
Sibling-GRPO	0.1204	0.1640	0.1335	**0.1749**
Joint	**0.1217**	**0.1641**	**0.1340**	0.1746

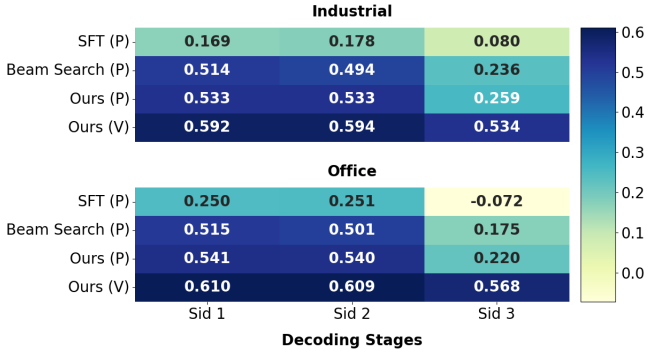


Figure 3: Spearman Correlation ρ of Probability(P) and Value (V) Signals with Ground-truth Rewards.

achieves a 4.0% relative improvement in HR@3 and a 4.3% relative improvement in NDCG@10 on *Industrial*. On *Office*, the gain is more pronounced, reaching a 10.4% relative improvement in HR@3. These consistent gains indicate that value-guided, budgeted exploration can reliably surface higher-reward SID candidates under strict prefix constraints, while tree-structured advantage reinforcement strengthens training by concentrating gradients on the branching decisions that actually differentiate sibling items.

5.3 Online Performance

To validate V-STAR in a real business environment, we conduct online A/B testing for 5 days on 5% of live request traffic on *Video Channels*. We use `GMV` (Gross Merchandise Volume) as the primary metric, i.e., the total transaction value attributed to advertising recommendations and a core indicator of commercial revenue. We compare against the *BeamSearch+GRPO* baseline. The online results show that V-STAR achieves a **1.23%** relative improvement in `GMV` and a **1.87%** relative improvement in `GMV-Normal` (GMV from click/conversion-optimized ad). These gains suggest that value-aware candidate construction, together with Sibling-GRPO, helps V-STAR more reliably surface high-commercial-value items, thereby increasing transaction volume.

5.4 Ablation Study

We conduct targeted ablation studies to disentangle the contributions of (i) the *decoding strategy* used to construct candidate sets, and (ii) the *training objective* used to optimize the policy.

Impact of the Decoding Strategy. To isolate decoding effects, we fix the trained way and vary only the inference-time candidate construction method. On *Industrial* and *Office*, we compare

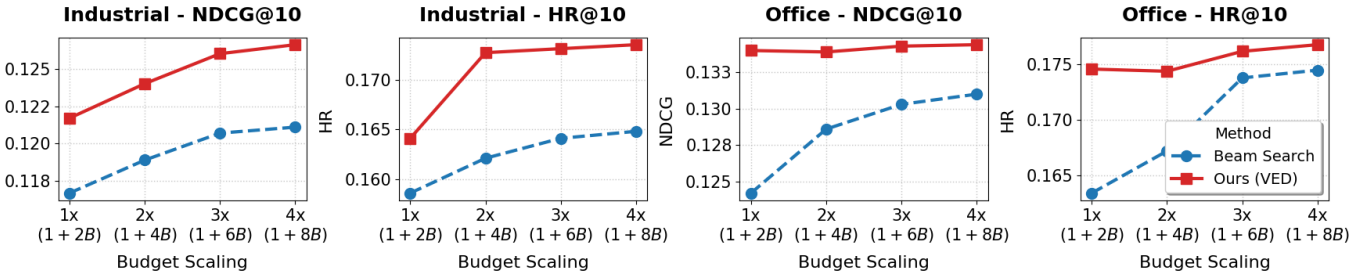


Figure 4: Training time scaling with decoding token budget.

Table 5: Case Study of different decoding strategies on *Office Products*. SIDs in red share prefixes with history items. SIDs in teal represent novel explorations. SIDs in green represent ground truth. VED shows higher candidate diversity and successfully discovers the ground-truth (GT) item in a novel branch (<a_20>), while baselines remain biased toward historical prefixes.

Context	Key Items & Candidate SIDs (For non-GT candidates, only the first SID are shown for simplicity)
History	Pencil Sharpener (<a_250>), Postcards (<a_225>), Bubble Mailer (<a_118>), Thank-you Stickers (<a_186>), Sharpie Pen (<a_102>)
GT Item	**ACCUTECK Digital Scale** (<a_20><b_181><c_107>)
Beam Search	Stickers (<a_186>), Label Tape (<a_117>), Sharpie Pen (<a_102>), Postcards (<a_225>), Bubble Mailer (<a_118>), Stickers (<a_186>), Label Printer (<a_117>), Address Labels (<a_186>), Shipping Labels (<a_117>), Label Tape (<a_117>) *(High redundancy: 6/10 items are historical prefixes; GT is missed)*
Top-K	Sharpie Pen (<a_102>), Thank-you Labels (<a_186>), Thank-you Stickers (<a_186>), Sharpie Marker (<a_102>), Sharpie Marker (<a_102>), Foam Board (<a_242>), EXPO Markers (<a_178>), Bubble Mailer (<a_118>), Glue Stick (<a_131>), File Folders (<a_204>) *(Prefix-biased: 6/10 items are historical prefixes; GT is missed)*
Ours (VED)	Thank-you Stickers (<a_186>), Thank-you Labels (<a_186>), Sharpie Pen (<a_102>), **ACCUTECK Digital Scale (GT)** (<a_20><b_181><c_107>), Round Labels (<a_186>), File Folders (<a_187>), Sharpie Marker (<a_102>), EXPO Markers (<a_178>), Poly Mailers (<a_118>), Glue (<a_242>) *(Finds GT via novel <a_20> branch while keeping diverse useful neighbors)*

(i) beam search, (ii) top-K (stochastic sampling from the K highest-probability tokens at each step), and (iii) our VED.[1] Table 2 shows that VED consistently achieves the best performance across datasets and metrics. Beam search is probability-dominated and may prune low-prior yet high-reward branches, while top-K improves diversity but yields less stable ranking due to untargeted exploration. In contrast, VED reallocates compute to a small set of decisive prefixes with high value and high uncertainty, mitigating early pruning and improving reachability of valuable long-tail items.

To isolate what drives the gain in VED, we keep the sampling procedure and candidate budget, and vary only the prefix priority score for selecting expansion nodes. Table 3 compares three rules: value-only (V_ϕ), entropy-only ($\mathcal{H}_\theta$), and the joint score $G(s)$ (Eq. 11) combining V_ϕ and $\mathcal{H}_\theta$. Value-only may over-exploit and miss low-prior yet high-reward branches, whereas entropy-only may over-explore high-uncertainty but low-reward regions. The joint score is consistently best, indicating that uncertainty mainly serves as an expansion gate: when value is high but confidence is already saturated, further expansion is redundant; uncertainty redirects budget to high-value yet under-resolved prefixes where exploration can still improve reachability.

[1]All methods enforce the same SID-validity constraint and return the same number of final items.

Impact of Sibling-GRPO. To evaluate the impact of the training objective, we fix VED candidate construction $C(x)$ and compare three RL objectives that differ only in how advantages are normalized on the VED-generated prefix tree: (i) GRPO with global group-based normalization over $C(x)$; (ii) Sibling-GRPO with sibling-relative normalization at shared-prefix branching points; and (iii) GRPO+Sibling-GRPO that jointly optimizes both losses. Table 4 shows that standard GRPO performs worst, as global normalization is dominated by inter-cluster variance and compresses intra-group advantages. Sibling-GRPO improves performance by aligning updates with the prefix-tree topology, focusing gradients on decisive branching nodes. Finally, the combined objective achieves the highest NDCG and HR. This suggests that global sequence-level alignment from GRPO and structure-aware, branch-level credit assignment from Sibling-GRPO are mutually reinforcing.

5.5 Further Analysis

How well do probability and value align with ground-truth reward across SID levels? To answer this, we form a candidate pool $C(x)$ of 64 SIDs for each query x by temperature sampling from 3 different models: SFT model, RL model trained with beam search+standard GRPO, and our V-STAR model. The temperature is set to 1.5 to cover various probability range. At each level $\ell \in \{1, 2, 3\}$, we compute Spearman correlation ρ between

Table 6: Diversity and max reward of candidate sets.

Method	Industrial		Office	
	Div(C) ↑	MaxReward ↑	Div(C) ↑	MaxReward ↑
Beam search	0.7949	0.2303	0.8624	0.2321
Top-K	0.8089	0.2246	**0.8856**	0.2160
Ours (VED)	**0.8167**	**0.2475**	0.8852	**0.2473**

prefix log-probability $\log \pi_\theta(y_{\le \ell} \mid x)$ and reward $R(x, y_{\le \ell})$, where $R(x, y_{\le \ell})$ is the average reward of candidate in $C(x)$ that share prefix $y_{\le \ell}$. For V-STAR, we also compute ρ between value prediction $V_\phi(x, y_{\le \ell})$ and $R(x, y_{\le \ell})$. We report ρ averaged over queries in Fig. 3. From the figure we can see that: (i) the predicted value $V_\phi(x, y_{\le \ell})$ aligns best with $R(x, y_{\le \ell})$, (ii) for probability–reward alignment, V-STAR > beam search+GRPO > SFT, indicating that RL improves the probability landscape and V-STAR further enhances such improvement, (iii) the correlation gap is largest at ℓ=3, where semantically similar leaf items reduce discriminability and weaken likelihood–reward correlation, while the value model remains strongly reward-aligned.

Does VED increase candidate-set diversity without sacrificing quality? We measure candidate-set diversity in SID space by calculating the average pairwise dissimilarity in candidate pool $C(x)$. For two SIDs v_i and v_j (each with L tokens), we define their similarity as the normalized length of the longest common prefix (LCP), i.e., $\mathrm{Sim}_{\mathrm{SID}}(v_i, v_j) = \mathrm{LCP}(v_i, v_j)/L$. Diversity is then computed as the average of $1 - \mathrm{Sim}_{\mathrm{SID}}(v_i, v_j)$ over all candidate pairs in $C(x)$, where a larger value indicates fewer near-duplicate SIDs and broader coverage across SID branches. To ensure diversity is not obtained by sacrificing strong candidates, we also report the best-in-set reward $\max_{v \in C} R(v)$, averaged over test queries. Three decoding strategies are tested: beam search, top-K, and our VED. Table 6 reports results with $|C(x)| = 64$. VED yields the highest (or matched-highest) diversity across datasets while consistently improving the best-in-set reward, indicating reduced SID redundancy and better discovery of high-reward candidates without trading off candidate quality. A detailed case study is given in Table 5.

Does VED exhibit more favorable scaling property than beam search under training-time decoding budgets? We compare VED with beam search under matched training-time decoding budgets. Specifically, we scale the decoding token budget by the number of model-evaluated tokens during candidate construction (excluding prompt prefill). For L=3 with beam width $B = 16$, the 1× budget is $1+2B$, and we proportionally expand it to 2× ($1+4B$), 3× ($1+6B$), and 4× ($1+8B$). Similarly, VED is constrained to the same budgets at each scale. Figure 4 shows that VED consistently outperforms beam search across budgets and metrics. On *Industrial*, VED yields larger gains at low budgets (1×→2×), whereas widening the beam provides smaller increments, indicating higher compute efficiency from value-guided expansion. On *Office*, VED maintains a clear margin at every scale, and 1× VED often matches or exceeds 3×–4× beam search. This suggests that value-guided expansion is more compute-efficient than indiscriminately widening the beam. Overall, VED converts additional training-time decoding compute into larger improvements than likelihood-driven widening by reducing redundant expansions and prioritizing high-value branches.

6 Conclusion

This paper studies candidate construction for Semantic-ID generative recommendation, where probability-driven decoding (e.g., beam search) tends to over-exploit high-probability sibling branches and provides limited reward contrast for RL. We propose V-STAR, which couples Value-Guided Efficient Decoding (VED) for budgeted candidate construction with Sibling-GRPO for structure-aware policy optimization. VED allocates a fixed decoding budget to a small set of high-potential branching points, improving reachability and candidate diversity without exhaustive tree search. Sibling-GRPO leverages the prefix-tree structure to compute sibling-relative learning signals, stabilizing optimization under prefix coupling. Extensive experiments show that V-STAR consistently improves over strong generative baselines under strict token budgets, and additional analyses verify better value–reward alignment and more effective use of extra decoding compute.

References

[1] Thomas Anthony, Zheng Tian, and David Barber. 2017. Thinking Fast and Slow with Deep Learning and Tree Search. In *Advances in Neural Information Processing Systems (NeurIPS)*. 5360–5370.
[2] Keqin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yanchen Luo, Chong Chen, Fuli Feng, and Qi Tian. 2025. A bi-step grounding paradigm for large language models in recommendation systems. *ACM Transactions on Recommender Systems* 3, 4 (2025), 1–27.
[3] Keqin Bao, Jizhi Zhang, Yang Zhang, Xinyue Huo, Chong Chen, and Fuli Feng. 2024. Decoding matters: Addressing amplification bias and homogeneity issue in recommendations for large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 10540–10552.
[4] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to Rank: From Pairwise Approach to Listwise Approach. In *Proceedings of the International Conference on Machine Learning (ICML)*. 129–136.
[5] Ben Chen, Xian Guo, Siyuan Wang, Zihan Liang, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, et al. 2025. Onesearch: A preliminary exploration of the unified end-to-end generative framework for e-commerce search. *arXiv preprint arXiv:2509.03236* (2025).
[6] Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, and Ed H Chi. 2019. Top-k off-policy correction for a REINFORCE recommender system. In *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*. 456–464.
[7] Yuxin Chen, Junfei Tan, An Zhang, Zhengyi Yang, Leheng Sheng, Enzhi Zhang, Xiang Wang, and Tat-Seng Chua. 2024. On softmax direct preference optimization for recommendation. *Advances in Neural Information Processing Systems (NeurIPS)* 37 (2024), 27463–27489.
[8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
[9] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*. 191–198.
[10] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
[11] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*. 299–315.
[12] Kartik Goyal, Graham Neubig, Chris Dyer, and Taylor Berg-Kirkpatrick. 2018. A continuous relaxation of beam search for end-to-end training of neural sequence models. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 32. 3045–3052.
[13] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).

[14] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations (ICLR)*.

[15] Minjie Hong, Zetong Zhou, Zirun Guo, Ziang Zhang, Ruofan Hu, Weinan Gan, Jieming Zhu, and Zhou Zhao. 2025. Generative Reasoning Recommendation via LLMs. *arXiv preprint arXiv:2510.20815* (2025).

[16] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952* (2024).

[17] Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, et al. 2019. Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2368–2375.

[18] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *IEEE International Conference on Data Mining (ICDM)*. IEEE, 197–206.

[19] Levente Kocsis and Csaba Szepesvári. 2006. Bandit based monte-carlo planning. In *Proceedings of the European Conference on Machine Learning (ECML)*. Springer, 282–293.

[20] Xiaoyu Kong, Leheng Sheng, Junfei Tan, Yuxin Chen, Jiancan Wu, An Zhang, Xiang Wang, and Xiangnan He. 2025. Minionerec: An open-source framework for scaling generative recommendation. *arXiv preprint arXiv:2510.24431* (2025).

[21] Jinming Li, Wentao Zhang, Tian Wang, Guanglei Xiong, Alan Lu, and Gerard Medioni. 2023. GPT4Rec: A generative framework for personalized recommendation and user interests interpretation. *arXiv preprint arXiv:2304.03879* (2023).

[22] Shuai Li, Xinyu Luo, Xiang Wang, Yongfeng Zhang, and Tat-Seng Chua. 2023. Large Language Models for Generative Recommendation: A Survey and Visionary Discussions. *arXiv preprint arXiv:2309.01157* (2023).

[23] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[24] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)* 35 (2022), 27730–27744.

[25] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[26] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems (NeurIPS)* 36 (2023), 10299–10315.

[27] Rajkumar Ramamurthy, Prithviraj Ammanabrolu, Kianté Brantley, Jack Hessel, Rafet Sifa, Christian Bauckhage, Hannaneh Hajishirzi, and Yejin Choi. 2023. Is Reinforcement Learning (Not) for Natural Language Generation?. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. 844–862.

[28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[29] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

[30] Amit Sharma, Hua Li, Xue Li, and Jian Jiao. 2024. Optimizing novelty of top-k recommendations using large language models and reinforcement learning. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 5669–5679.

[31] Chufan Shi, Haoran Yang, Deng Cai, Zhisong Zhang, Yifan Wang, Yujiu Yang, and Wai Lam. 2024. A thorough examination of decoding methods in the era of llms. *arXiv preprint arXiv:2402.06925* (2024).

[32] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. 2017. Mastering the game of go without human knowledge. *nature* 550, 7676 (2017), 354–359.

[33] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450.

[34] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems (NeurIPS)* 27 (2014).

[35] Jiaxi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*. 565–573.

[36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[37] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 3245–3254.

[38] Zhipeng Wei, Kuo Cai, Junda She, Jie Chen, Minghao Chen, Yang Zeng, Qiang Luo, Wencong Zeng, Ruiming Tang, Kun Gai, et al. 2025. Oneloc: Geo-aware generative recommender systems for local life service. *arXiv preprint arXiv:2508.14646* (2025).

[39] Yepeng Weng, Qiao Hu, Xujie Chen, Li Liu, Dianwen Mei, Huishi Qiu, Jiang Tian, and Zhongchao Shi. 2025. Traversal Verification for Speculative Tree Decoding. *arXiv preprint arXiv:2505.12398* (2025).

[40] Ronald J Williams. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning* 8 (1992), 229–256.

[41] Sam Wiseman and Alexander M Rush. 2016. Sequence-to-sequence learning as beam-search optimization. *arXiv preprint arXiv:1606.02960* (2016).

[42] Haibo Xing, Hao Deng, Yucheng Mao, Jinxin Hu, Yi Xu, Hao Zhang, Jiahao Wang, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, et al. 2025. Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. *arXiv preprint arXiv:2508.15308* (2025).

[43] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[44] Weiqin Yang, Bohao Wang, Zhenxiang Xu, Jiawei Chen, Shengjia Zhang, Jingbang Chen, Canghong Jin, and Can Wang. 2026. BEAR: Towards Beam-Search-Aware Optimization for Recommendation with Large Language Models. *arXiv preprint arXiv:2601.22925* (2026). https://arxiv.org/abs/2601.22925 Preprint.

[45] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, et al. 2025. Sparse meets dense: Unified generative recommendations with cascaded sparse-dense representations. *arXiv preprint arXiv:2503.02453* (2025).

[46] Shunyu Yao, Dian Yu, Jeffrey Zhao, Tom Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems (NeurIPS)* 36 (2023), 11809–11822.

[47] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. SoundStream: An End-to-End Neural Audio Codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 30 (2021), 495–507.

[48] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. 2024. Actions Speak Louder than Words: Trillion-parameter Sequential Transducers for Generative Recommendations. In *Proceedings of the International Conference on Machine Learning (ICML)*. 58484–58509.

[49] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[50] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *IEEE International Conference on Data Engineering (ICDE)*. 1435–1448.

[51] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. 2025. Onerec-v2 technical report. *arXiv preprint arXiv:2508.20900* (2025).

A Algorithm Pseudocode

We provide detailed pseudocode for the two core components of V-STAR: Value-Guided Efficient Decoding (Algorithm 1) and Sibling-GRPO training (Algorithm 2).

Algorithm 1: Value-Guided Efficient Decoding (VED)

Input: Policy π_θ, value model V_ϕ, prefix tree $\mathcal{T}$, budget B, UCB weight β, exploration coefficient λ
Output: Candidate set $C(x) \subset \mathcal{V}^L$

$\mathcal{T} \leftarrow \text{BeamSearch}(\pi_\theta, x, W)$ `// Initialize with beam width W`
Evaluate $\{V_\phi(s), \mathcal{H}_\theta(s), G(s)\}$ for all nodes in $\mathcal{T}$
while $\text{Cost}(\mathcal{T}) \le B$ **do**
  `// Selection: UCB-style traversal`
  $s^* \leftarrow$ root
  while s^* *is not a leaf* **do**
    $s^* \leftarrow \arg\max_{c \in \text{children}(s^*)} \left[G(c) + \beta \sqrt{\frac{\ln(N_{\text{root}}+1)}{N(c)+1}} \right]$
  end
  `// Gated expansion`
  if $G(s^*) \ge \bar{G}_{\text{depth}(s^*)}$ **then**
    Sample child c from unexpanded valid tokens under $\pi_\theta(\cdot \mid s^*)$
    Add c to $\mathcal{T}$; evaluate $\{V_\phi(c), \mathcal{H}_\theta(c), G(c)\}$
  end
  `// Backpropagation`
  Update $N(\cdot)$ and $\bar{G}_\ell$ along ancestor path of c
end
$C(x) \leftarrow$ top-valued complete sequences (depth L) from $\mathcal{T}$
return $C(x)$

Algorithm 2: V-STAR Training with Sibling-GRPO

Input: Policy π_θ, old policy $\pi_{\theta_{\text{old}}}$, candidate set $C(x)$, reward R
Output: Updated policy π_θ

for *each training context* x **do**
  Obtain $C(x)$ via VED (Algorithm 1)
  Compute rewards $\{R(x,y)\}_{y \in C(x)}$
  `// Sibling-GRPO: compute advantages per depth`
  for *depth* $\ell = 1, \ldots, L$ **do**
    foreach *parent prefix* $h \in \mathcal{V}^{\ell-1}$ *with* $|\mathcal{S}(h)| \ge 2$ **do**
      foreach *sibling node* $v \in \mathcal{S}(h)$ **do**
        $\bar{R}(x;h,v) \leftarrow \frac{1}{|\mathcal{G}(h,v)|} \sum_{y \in \mathcal{G}(h,v)} R(x,y)$
        $A_{\text{node}}(x;h,v) \leftarrow \frac{\bar{R}(x;h,v) - \mu_h}{\sigma_h + \epsilon}$
      end
    end
  end
  `// Policy update (joint objective)`
  $\mathcal{L} \leftarrow \mathcal{J}_{\text{GRPO}}(\theta) + \mathcal{J}_{\text{sib}}(\theta)$
  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
end

B Implementation Details

Our implementation is publicly available at https://github.com/Yara-HYR/V-star.git. Table 7 summarizes the hyperparameter configuration used in all experiments. Both datasets (Industrial and Office) share identical settings.

Table 7: Hyperparameter configuration for V-STAR.

Hyperparameter	Value
Backbone	Qwen2.5-1.5B
Learning rate (policy & value)	1×10^{-5}
Batch size (per GPU) × GPUs	64×8
Gradient accumulation steps	2
GRPO group size G	16
KL regularization coefficient	1×10^{-3}
Exploration coefficient λ (Eq. 11)	0.1
UCB weight β (Eq. 13)	1.0
Hierarchical weights w_l (Eq. 8)	[0.3, 0.5, 1.0]
Discount factor γ (Eq. 9)	0.99
VED initialization beam width	8
SID depth L / codebook size $\lvert\mathcal{V}\rvert$	3 / 256

Sensitivity analysis. We study the sensitivity of three key hyperparameters on the Industrial dataset. (1) **Exploration coefficient** λ (Eq. 11): optimal at 0.1; performance varies less than 0.6% across the range [0.05, 0.2], indicating low sensitivity. (2) **UCB weight** β (Eq. 13): optimal at 1.0; performance varies less than 0.5% across [0.5, 2.0]. (3) **Hierarchical weights** w_l (Eq. 8): optimal at [0.3, 0.5, 1.0] (monotonically increasing). The alternative [0.1, 0.3, 1.0] degrades by less than 0.3%; uniform weighting [1.0, 1.0, 1.0] or terminal-only [0.0, 0.0, 1.0] degrades by approximately 1–2%, confirming the benefit of dense hierarchical supervision with increasing emphasis on deeper levels.

C Training Overhead

Table 8 reports the per-step training overhead of V-STAR relative to the GRPO baseline.

Table 8: Per-step training overhead: Baseline (BeamSearch + GRPO) vs. V-STAR.

Metric	Baseline	V-STAR	Ratio
FLOPs / step	1978 TF	2218 TF	1.12×
Peak GPU memory	32–36 GB	34–39 GB	1.07×
Wall-clock / step	~11 s	~14 s	1.27×

The overhead is modest: 12% additional FLOPs and 27% wall-clock time per step. Sibling-GRPO modifies only the loss computation with negligible cost; VED rollouts account for the majority of the increase. Critically, VED is employed *only during training* for constructing high-quality candidate sets. At inference time, V-STAR uses standard beam search, incurring *identical latency and throughput* to the baseline.

Table 9: Cross-backbone results on Industrial. Baseline denotes BeamSearch + GRPO with the same backbone.

Backbone	HR@10		NDCG@10	
	Baseline	V-STAR	Baseline	V-STAR
GPT-2 (70M)	0.0922	0.1235	0.0750	0.0987
Qwen2.5-0.5B	0.1553	0.1630	0.1114	0.1209
Qwen2.5-1.5B	0.1586	0.1641	0.1167	0.1217
Qwen2.5-3B	0.1669	0.1721	0.1227	0.1240

D Cross-Backbone Generalization

To verify that V-STAR is architecture-agnostic, we evaluate it across four backbone models of varying capacity on the Industrial dataset (Table 9). V-STAR consistently improves both HR@10 and NDCG@10 across all backbones. The gains are largest on GPT-2 (70M), where the weaker base policy has limited coverage of the SID space and benefits most from value-guided exploration. These results confirm that V-STAR's improvements are not contingent on a specific model architecture or scale.